NUMA Memory Policy for Fusion Compiler–Class Physical Design Jobs

A design and operations white paper for the `NUMA-EDA-Bench` project

Authors: Project notes distilled from implementation and experiments in `Physical-Design-Algorithms-Implementation`
Scope: Job-local DRAM / NUMA placement for ASIC/SoC PD workloads on multi-socket Linux compute farms
Companion code: `NUMA-EDA-Bench/` (microbench, before/after harness, production wrapper, LinkedIn demo)
Sibling project: `PD Job Acceleration/` (tmpfs + rsync for filesystem placement)

Abstract

Physical-design (PD) tools such as Synopsys Fusion Compiler, Cadence Innovus, and large-scale STA/extraction engines are routinely described as CPU-bound. On dual- and quad-socket farm nodes, a large fraction of wall-clock time can instead be spent waiting on **remote DRAM**: cores on one NUMA node touching pages that live on another. The interconnect (Intel UPI/QPI, AMD Infinity Fabric) adds latency and steals bandwidth under load. `top` still shows 100% CPU; turnaround does not improve with "more cores."

This white paper presents a practical, license-preserving acceleration pattern:

1. Discover NUMA topology (`numactl -H`, sysfs).
2. For one fat PD job that **fits in one node's free RAM**, pin CPUs and bind memory to the **same** node: `numactl --cpunodebind=N --membind=N <tool> …`
3. Measure before/after with a STREAM + pointer-chase + graph-walk microbench (and, on real silicon, with `numastat` / wall time of a real stage).
4. Refuse hard `membind` when free memory is insufficient; fall back to `--preferred` or resize the job/machine.

We describe the Linux first-touch model, failure modes, a before/after harness modeled on the repository's tmpfs+rsync experiments, and how to defend the approach in design reviews. An **emulated remote tax** exists only so single-node hosts can smoke-test the harness; those numbers are **not** results and must not be published as speedups. The central claim is not a new PD algorithm; it is that **memory placement is part of turnaround**, and that claims require topology plus measurement on real multi-socket silicon (or real tool wall time).

Table of contents

---

1. Motivation and industry context

1.1 The multi-socket PD farm

A modern PD host is often a **2-socket (2S)** or **4-socket** Xeon/EPYC machine with hundreds of GB of DRAM. Each socket is a **NUMA node**: it owns a set of cores and a set of memory controllers. Cross-node loads are legal but not free.

Fusion Compiler–class jobs:

- allocate multi-GB–TB working sets (netlist, timing graph, routing DB),
- stream and chase pointers through those structures,
- spawn many worker threads.

If threads run on node 0 while the hot heap sits on node 1, every miss pays interconnect latency and contends with other jobs' remote traffic.

1.2 Misdiagnosis is common

Teams respond with:

- more cores / higher thread counts,
- larger machines,
- or algorithm-level tuning,

when the binding constraint is **memory locality**. Conversely, hard-binding a job that does not fit in one node's free RAM produces OOM or reclaim storms that look like "`numactl` broke my run."

This project exists to make the correct middle path explicit, measurable, and safe—parallel to how `PD Job Acceleration` made tmpfs+rsync explicit for I/O.

---

2. Problem statement

> **Given** a PD tool command on a multi-socket Linux host,
> **accelerate wall-clock turnaround** by ensuring CPU threads and DRAM pages share a NUMA node,
> **without** changing the tool binary, licenses, or functional results,
> **while** avoiding OOM from hard `membind` on undersized nodes.

Success criteria:

1. **Functional equivalence** — same Tcl, same deliverables.
2. **Measurable locality win** — higher STREAM-like bandwidth, lower chase / graph latency, lower wall time on real stages when remote was the bound.
3. **Bounded risk** — refuse unsafe `membind`; document topology.
4. **Operability** — one wrapper command; clear before/after harness.

---

3. NUMA in one page

NUMA = Non-Uniform Memory Access.

Access	Typical cost
Local DRAM (same node)	baseline latency / full controller BW
Remote DRAM (other node)	+latency, −effective BW, interconnect contention

Topology discovery:

```
numactl -H
lscpu | grep NUMA
ls /sys/devices/system/node
```

Distance matrices in `numactl -H` encode relative hop costs (lower = closer).

---

4. Why first-touch creates remote fills

Linux often places a newly written anonymous page on the **node of the CPU that first touched it** (first-touch).

Canonical PD failure mode:

1. Master thread on node 0 allocates and zeros a huge array / DB arena.
2. Or init runs on node 1 while workers later run on node 0.
3. The OS load-balances threads across sockets while the heap stays put.

AutoNUMA / migrate-on-fault can move pages, but slowly and imperfectly for short phases and huge stable working sets. Overnight FC with a hot heap still suffers if policy is left to chance.

`membind` + `cpunodebind` remove ambiguity for production runs.

---

5. Design principles

1. **Same node for CPUs and pages** when the job fits.
2. **Topology before policy** — 1-node machines gain nothing from bind theater.
3. **Hard bind only with headroom** — peak RSS × safety factor < node MemFree.
4. **Measure, don't folklore** — before/after harness + `numastat -p`.
5. **Separate I/O locality from DRAM locality** — tmpfs and `numactl` solve different bottlenecks; both can apply on the same job.

---

6. The policy: `numactl` bind

6.1 DeepSeek-style recipe (one fat FC)

```
numactl --cpunodebind=0 --membind=0 fc_shell -f run.tcl
```

Flag	Meaning
`--cpunodebind=0`	schedule only on node 0 CPUs
`--membind=0`	allocate only from node 0 memory
`--localalloc`	prefer the touching CPU's node
`--preferred=0`	prefer node 0, spill if needed
`--interleave=all`	stripe pages (sometimes for huge shared heaps)

6.2 Soft vs hard

- **`membind`** — hard; fails or stalls if the node is exhausted.
- **`preferred`** — soft; safer when RSS is uncertain.
- **`interleave`** — can help when the working set intentionally spans the machine; often hurts a single mid-size FC that would fit on one node.

---

7. Relationship to tmpfs + rsync

Concern	Project	Lever
NFS / disk latency, tiny-file storms	`PD Job Acceleration`	tmpfs hot tier + rsync durability
Remote DRAM / UPI contention	`NUMA-EDA-Bench`	`numactl` CPU+mem bind

A job can be **I/O-bound on NFS** and **NUMA-bound on DRAM** in different phases. Profile each; apply the matching lever.

---

8. Experimental methodology

8.1 Microbench (`numa_mem_bench`)

FC-class proxy (no EDA license required):

1. **STREAM triad** — `a[i]=b[i]+s*c[i]` bandwidth (GiB/s).
2. **Pointer chase** — random cycle over a large index array (ns/hop).
3. **Graph walk** — degree-4 CSR-like random edges (timing/netlist proxy).
4. **wall_proxy** — composite score (lower better).

Machine-readable:

```
./build/numa_mem_bench --json --bytes 512M --threads 4
```

8.2 Before/after harness (`examples/compare_numa.sh`)

Modeled on `PD Job Acceleration/examples/compare_accel.sh` and `compare_pd_farm_io.sh`.

Hardware path (≥2 nodes):

Leg	Command
BEFORE	`numactl --cpunodebind=0 --membind=1 ./build/numa_mem_bench …`
AFTER	`numactl --cpunodebind=0 --membind=0 ./build/numa_mem_bench …`

Emulated path (1 node — cloud / laptop):

Leg	Command
BEFORE	`./build/numa_mem_bench --emulate-remote --remote-bw-mult=0.55 --remote-lat-mult=1.75 …`
AFTER	local (optionally under `membind=0`)

Emulation is **explicitly labeled**, analogous to `--nfs-us` in the farm I/O suite: it lets the harness and documentation stay honest on single-node hosts while still producing a BEFORE/AFTER artifact. Design-review claims for a farm should cite a **hardware** run on a 2S/4S node.

8.3 What we do not claim

- Emulated ratios are not silicon measurements.
- Microbench ≠ full Fusion Compiler wall time.
- Functional PD QoR is unchanged by `numactl`; only resource placement changes.

---

9. Results

9.0 Claims gate

Mode	When	May you cite ratios publicly?
`hardware`	≥2 NUMA nodes; remote `membind` vs local `membind`; median of ≥3 trials	Yes, as **microbench** (still not FC wall time)
`emulated`	1-node host; artificial `--emulate-remote` tax	**No**

Every run writes `examples/compare_results/CLAIM_GATE.txt`.

9.1 This repository cloud host (1 NUMA node) — harness smoke only

Environment:

- 1 socket / 1 NUMA node / 4 CPUs
- Mode: **emulated** → `CLAIM_GATE=DO_NOT_CITE`

This host **cannot** measure remote DRAM. Running `compare_numa.sh` here only proves the harness runs. Triad especially is noisy on shared VMs (AFTER triad can land below BEFORE even when latency metrics move the expected way). That is measurement noise, not an anti-NUMA result, and not a publishable win.

Do not copy emulated SUMMARY ratios into LinkedIn, slides, or customer mail.

9.2 What a citeable hardware result looks like

On a real dual-socket machine, re-run:

```
TRIALS=5 BYTES=1G THREADS=$(nproc) ./examples/compare_numa.sh
cat examples/compare_results/CLAIM_GATE.txt   # must say CITEABLE=yes
cat examples/compare_results/SUMMARY.txt
```

Replace this section with that SUMMARY (and ideally FC stage wall time + `numastat -p`). Until then, this paper's "result" is the **methodology**, not a farm speedup number.

Order-of-magnitude expectations from literature / field practice (not measured here):

Metric	Remote vs local (typical shape)
STREAM-like bandwidth	remote often lower under load
Pointer-chase latency	remote higher
FC stage wall time	improves only if the stage was interconnect-bound and RSS fits

9.3 How to read `SUMMARY.txt`

- **triad** — higher better (bandwidth); check **triad spread** across trials
- **chase_ns / graph_ns** — lower better

- **wall_proxy** — synthetic composite only; **never** call it FC wall-clock
- Ratios are labeled AFTER/BEFORE or BEFORE/AFTER accordingly
- If `DO_NOT_CITE` is present, stop — do not publish

---

10. When the pattern helps—and when it does not

Situation	Likely outcome
2S/4S box, one FC, RSS fits one node	High — primary target
Already 1 NUMA node	None from bind; harness uses emulation only
RSS > node free RAM + hard membind	Harm — OOM / thrash
Bound is NFS / disk	Fix I/O first (`PD Job Acceleration`)
Many small jobs	Spread jobs across nodes (one job per node)
Intentionally machine-spanning heap	Consider interleave / preferred, not hard single-node membind

---

11. Production wrapper and safety

`scripts/run_eda_numactl.sh`:

```
./scripts/run_eda_numactl.sh 0 fc_shell -f run.tcl
```

Behavior:

1. Verifies `numactl` and node existence.
2. Prints node MemFree and full `numactl -H` into the log stream.
3. Refuses `membind` if MemFree < 8 GiB unless `FORCE=1`.
4. `exec`s `numactl --cpunodebind=N --membind=N …`.

Operational checklist:

1. `numactl -H` / free memory per node.
2. Estimate peak RSS (prior runs, `numastat`, peak from farm accounting).
3. Bind only with headroom.
4. Log policy into the run manifest.
5. Optional live check: `numastat -p $(pgrep -n fc_shell)`.

---

12. Defending the approach (review / interview)

Claim: “We improved FC turnaround with `numactl`.”

Defense structure:

1. **Topology** — show `numactl -H` (multi-node).
2. **Hypothesis** — remote fills / first-touch scatter.
3. **Measurement** — BEFORE remote vs AFTER local microbench; ideally stage wall times + `numastat`.
4. **Safety** — MemFree vs peak RSS; refuse unsafe membind.
5. **Non-claims** — not a QoR change; not a substitute for I/O tiering.
6. **Sibling** — I/O locality handled by tmpfs+rsync project when NFS-bound.

Interview one-liner:

> Same silicon, better memory policy: pin FC CPUs and pages to one NUMA node when the working set fits—because remote DRAM is a tax `top` cannot see.

---

13. Limitations and future work

- Emulated remote tax is a model (calibrated defaults 0.55× BW / 1.75× lat).
- No licensed FC binary in CI — microbench + wrapper only.
- Future: cgroup/cpuset farm integration; automatic RSS vs MemFree advisor; `perf c2c` recipes; hugepage interaction notes.

---

14. Conclusion

Multi-socket PD farms hide a class of performance bugs in plain sight: **CPU busy, memory remote**. The DeepSeek-style `numactl --cpunodebind --membind` recipe is the correct first response when a single Fusion Compiler–class job fits in one node's RAM. This repository turns that recipe into a before/after experiment, a production wrapper, a white paper, and a visual LinkedIn demo— the DRAM-locality counterpart to tmpfs+rsync for filesystem locality.

Appendix A: quick-start commands

```
cd NUMA-EDA-Bench
./scripts/numa_report.sh
make -j
./examples/compare_numa.sh
cat examples/compare_results/SUMMARY.txt

# Production-style wrap
./scripts/run_eda_numactl.sh 0 fc_shell -f run.tcl

# LinkedIn GIF
python3 demo/gen_numa_linkedin_gif.py
```

```
# White paper PDF
python3 docs/build_whitepaper_pdf.py
```

Appendix B: glossary

Term	Meaning
NUMA	Non-Uniform Memory Access
UPI / IF	Intel Ultra Path Interconnect / AMD Infinity Fabric
first-touch	Page placed on the node of the CPU that first writes it
membind	Hard memory-node binding (`numactl`)
cpunodebind	Restrict threads to a node’s CPUs
wall_proxy	Composite microbench score (lower better)
emulate-remote	Labeled software remote-DRAM tax for 1-node hosts